

# **Higher National Diploma in Software Engineering**

## **PDSA Project Report**

### **Student Performance Ranking System**

**25.2F**



**Submitted by**

W.L.J. Hansani

GAHDSE252F-027



**School of computing and Engineering  
National Institute of Business Management  
GALLE**

## Table of Contents

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>INTRODUCTION .....</b>                                  | <b>3</b> |
| <b>PROBLEM STATEMENT .....</b>                             | <b>3</b> |
| <b>LITERATURE REVIEW .....</b>                             | <b>4</b> |
| 1. Moodle Learning Management System (LMS) .....           | 4        |
| 2. Canvas LMS .....                                        | 4        |
| 3. Power BI Education dashboard .....                      | 5        |
| <b>PROPOSED SOLUTIONS .....</b>                            | <b>5</b> |
| <b>SYSTEM FUNCTIONALITIES .....</b>                        | <b>6</b> |
| <b>SELECTED DATA STRUCTURES &amp; JUSTIFICATIONS .....</b> | <b>7</b> |
| AVL Tree Justification .....                               | 7        |
| Singly Linked List Justification .....                     | 7        |
| <b>NOVEL FEATURES .....</b>                                | <b>7</b> |
| <b>TECHNOLOGIES USED .....</b>                             | <b>8</b> |
| <b>SELLING POINTS OF THE APPLICATION .....</b>             | <b>8</b> |
| <b>INDIVIDUAL REFLECTION .....</b>                         | <b>8</b> |
| <b>GitHub LINK .....</b>                                   | <b>9</b> |
| <b>CONCLUSION .....</b>                                    | <b>9</b> |
| <b>REFERANCES .....</b>                                    | <b>9</b> |

## INTRODUCTION

Educational institutions and universities need an easy and efficient system to manage, analyze and rank student performance within multiple modules and batches. Based on the number of students increment, it's becoming exceedingly difficult to keep track of their results and identify how each student performs recently.

The following report shows analytics of Scholastic Insight, a Student Performance Ranking System. The system includes two main data structures to optimize the user flows.

1. **AVL Tree** - This is a data structure we used to rank students based on their performance. This AVL tree is a self-balancing binary search tree (BST) because of that it enables very fast data searching even as the volume of data increases. Its search time complexity is  $O(\log n)$ .
2. **Singly Linked List** - A Singly Linked List is used to store a student's past and current results; we analyze whether their performance is improving, declining, or remaining at the same level.

## PROBLEM STATEMENT

Educational institutions have to manage large volumes of data related to students, such as assignment marks, exam scores, attendance percentages, and the modules for which students are registered. As the number of students increases, it becomes difficult to carry out many tasks manually. Some of these are listed below.

- Identify top performing students
- Identify low performing students
- Tracking changes in student performance overtime
- Identify students at academic risk

In a traditional system, data must be re-sorted whenever a ranking is required, which can become slow as the number of students increases. And also without tracking a student's past results, it is difficult for the teacher to identify how their performance has changed over time.

A main problem arising from the increase in the number of students is the lack of a suitable automated system to efficiently manage learning and assessment activities within the institution. Because of this, it is difficult to do the following:

- Difficult to quickly rank students based on their marks
- Difficult to track students' past results and progress
- Difficult to update students rankings efficiently without sorting the data again every time new data is added
- Difficult to visually display the data structures
- Difficulty to automatically identify students who are at academic risk

## LITERATURE REVIEW

### 1. Moodle Learning Management System (LMS)

This is a Learning Management System (LMS) widely used in schools and universities. It enables teachers to manage courses, assignments, student grades, and learning activities from a single location. Students can access online learning materials, submit assignments, and view their grades.

This system allows for the management of student learning activities and grades and it has certain limitations regarding student performance ranking. It does not offer real time student ranking using custom data structures like AVL trees as a core feature. And also it lacks visual representations of data structures and does not track student performance history using structures such as singly linked lists. Because of that this system does not automatically analyze whether a student's performance is improving, declining, or stable through the use of such custom data structures like our system.

### 2. Canvas LMS

Google Classroom is a cloud based learning platform that helps teachers manage classes, assignments, and student work. It features a simple interface, making it easy for both teachers and students to use. Teachers can create and share assignments, while students can access learning materials and submit their assignments online.

However Google classroom has certain limitations when it comes to analyzing student performance. It lacks a specific performance score formula to calculate student performance or a real time leaderboard to rank students. And also it does not automatically analyze whether a student's performance is improving, declining, or remaining stable over time.

### 3. Power BI Education dashboard

Power BI is a tool used to analyze and visualize data. It facilitates easy data understanding by providing dashboards, charts and reports. And also it helps in viewing student information and identifying patterns in their performance.

However Power BI has certain limitations. In sometime data must be imported manually and it does not use custom data structures such as AVL trees and singly linked lists for processing student data. And also paid licences are required to access some of its features.

In comparison our Scholastic Insight System uses custom data structures to rank students and track their performance history.

None of the systems we reviewed use self balancing tree structures such as AVL trees or custom singly linked lists to track real time student ranking and changes in student performance overtime. Databases level sorting is normally used for ranking or sorting in those systems and its time complexity is  $O(n \log n)$ . However we use AVL Trees and Singly Linked List as custom data structures and this enables more efficient management of students ranking and performance history.

## PROPOSED SOLUTIONS

The Scholastic Insight system we propose use two key data structures to address the identified problems.

- AVL Tree- Student data is stored based on their performance scores. This helps ranking students, quickly searching for a specific student and efficiently generating a ranking list.
- Singly Linked List- It is used to systematically store each student's past performance records. New performance records can be added quickly. And also by analyzing this history one can track whether a student's performance improving, declining or remaining over time.

### System Workflow

- The lecturer enters the student's assignment marks ,exam marks and attendance into the system.
- The system calculates the student's performance score
- The student's data added to AVL Tree which automatically balances the data
- The students' performance record is also added to a Singly Linked List
- The system visually displays both the AVL Tree and Singly Linked List
- The system generates a ranking list of students and checks each student's performance to track if it is improving, declining or stable.

## SYSTEM FUNCTIONALITIES

The system provides the following main functions:

- User Login-Admins, Lecturers and students can securely log into the system
- Batch and Module Management-Admins can create and manage batches and modules.
- Student Performance-Lecturers can enter student ' assignment marks ,exam marks , and attendance into the system.
- AVL Tree -Student performance records are added to an AVL Tree which automatically balances itself
- Student Ranking- Students are ranked based on their performance scores from the highest score to the lowest
- Student Search- Allows for the quick searching of student based on their performance score
- Risk Students-The system automatically identifies students as at risk if their score is below 50 or their attendance is below 70%
- AVL Tree Visualization- Visually displays the AVL Tree and its structure
- Performance History- Stores students' past performance records using a singly linked list enabling the system to track whether performance is improving ,declining or stable
- Student Dashboard-Allows students to view their performance ,modules,ranking and performance history
- Delete Records- Enables the deletion of a performance record from the AVL Tree , the tree automatically rebalances after the deletion.

## SELECTED DATA STRUCTURES & JUSTIFICATIONS

### AVL Tree Justification

- Fast Operations-The AVL Tree can add, search, and update student records quickly in  $O(\log n)$  time
- Automatically Balanced- The tree automatically keeps itself balanced when new student marks are added
- Easy Student Ranking – The system can quickly create a ranking list from the highest score to the lowest score

### Singly Linked List Justification

- Fast Record Adding- New performance records can be added quickly at the beginning of the list in  $O(1)$  time
- Performance History - It stores the student's latest and previous performance records in order
- Latest Score Change- The system can quickly compare the latest score with the previous score.
- Performance Trend- Identifies performance as Improving , Declining or stable

## NOVEL FEATURES

The system has four novel features

- Live AVL Tree Visualization- Shows the AVL Tree and student performance data visually on the dashboard
- Performance History Tracking – Use singly link list to store previous performance and identifying whether a student is improving, declining or stable
- Risk Student Detection-Automatically identify student with a score below 50 or attendance below 70%
- Fast Student Search-Use the AVL Tree to quickly find students by their performance score in  $O(\log n)$  time

## TECHNOLOGIES USED

- **Java-** Used for backend development and data structures
- **Spring Boot-** Used to create the backend and APIs
- **MongoDB-** Used to store system data
- **React.js-** Used to create the frontend user interface.
- **Vite-** used to run and build the frontend quickly
- **Gradle-** Used to manage backend dependencies and build the project

## SELLING POINTS OF THE APPLICATION

- Two Data Structures-use an AVL Tree for student ranking and singly linked list for performance history
- Fast performance-The AVL Tree keeps data balanced and allows fast searching
- Fast History Adding-New performance records can added quickly
- Visual Data structure-Shows the AVL Tree and Singly Linked List visually
- User roles-Provides separate access for admin, lecturer and student
- Risk Detection-Quickly identify students with low scores or attendance
- Modern Inaterfaces- Provides a simple, responsive and user friendly web interfaces

## INDIVIDUAL REFLECTION

In this project I was responsible for develop the admin part. My main tasks were managing batches, modules and other administrtrive tasks. The main challenge I face was connect and manage these functions correctly. I solve these problems by testing each functions and fixing errors step by step. In this project I improve my problems solving skills, system development and teamwork and gainedvaluable practical experience



## GitHub LINK

**Link :-** [https://github.com/jayaniwijemanna/Student-\\_performance\\_ranking-System.git](https://github.com/jayaniwijemanna/Student-_performance_ranking-System.git)

## CONCLUSION

This project shows how AVL Tree and Singly Linked List can be used to manage student performance efficiently. The system can rank students, track performance history, identify at risk students and search students records quickly. Using Spring Boot and React.js Scholastic Insight provides a simple and effective solution for managing and analyzing student performance.

## REFERANCES

- GeeksforGeeks. (2025b, December 20). *AVL Tree Data Structure*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/introduction-to-avl-tree/>
- GeeksforGeeks. (2025a, July 23). *Singly linked list problems*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/introduction-to-singly-linked-list/>
- *Spring boot*. (n.d.). Spring Boot. <https://spring.io/projects/spring-boot>
- *Home | Moodle.org*. (n.d.). <https://moodle.org/>